

Aplicaciones con microprocesadores y microcontroladores.

M. en C. Saul Villegas.

Universidad Autónoma de la Ciudad de México.

13 de septiembre de 2026

B.1

Flujo de desarrollo de firmware

B.1.1

Firmware

¿Qué es el firmware?

Definición

El firmware es el software que controla directamente el funcionamiento del hardware de un sistema embebido. Se almacena en la memoria no volátil del dispositivo y comienza a ejecutarse inmediatamente después del proceso de arranque.

Funciones principales

El firmware permite que el microcontrolador interactúe con el hardware del sistema.

- Configura los periféricos.
- Lee sensores.
- Controla actuadores.
- Implementa protocolos de comunicación.
- Ejecuta la lógica de control.

Concepto fundamental

El firmware constituye el vínculo entre el software y el hardware. Todo el código desarrollado durante este curso será firmware ejecutándose directamente sobre el microcontrolador RP2040.

B.1.2 Toolchain

¿Qué es una Toolchain?

Cadena de herramientas

Una *toolchain* es el conjunto de herramientas utilizadas para transformar el código fuente de un programa en código ejecutable para una arquitectura específica.

En sistemas embebidos, estas herramientas se ejecutan normalmente en una computadora y generan código para el microcontrolador donde se ejecutará el firmware.

Código fuente $\longrightarrow$ **Toolchain** $\longrightarrow$ **Firmware**

Cadena de transformación

La generación de firmware involucra diferentes herramientas especializadas:

- **Preprocesador:** procesa directivas y archivos de cabecera.
- **Compilador:** analiza y transforma el programa.
- **Ensamblador:** convierte ensamblador en código objeto.
- **Enlazador:** integra objetos y bibliotecas.
- **Utilidades:** inspeccionan o convierten los archivos generados.

`.c → .i → .s → .o → .elf → .uf2`

¿Qué ocurre dentro del compilador?

La compilación no consiste únicamente en traducir directamente un archivo de C a instrucciones de máquina. El compilador realiza diferentes fases de análisis y transformación.

Código fuente → Análisis léxico → Análisis sintáctico → Análisis semántico
→ Representación intermedia → Optimización → Generación de código

Front-end del compilador

Las primeras fases permiten comprender la estructura y significado del programa.

- **Análisis léxico:** identifica palabras reservadas, identificadores, operadores y constantes.
- **Análisis sintáctico:** verifica que los elementos formen estructuras válidas del lenguaje.
- **Análisis semántico:** comprueba tipos, declaraciones y compatibilidad de las operaciones.

Ejemplo

```
int resultado = a + 10;
```

Del lenguaje a una representación interna

Después de analizar el programa, el compilador genera una representación intermedia (*Intermediate Representation*, IR).

Esta representación permite separar el lenguaje utilizado por el programador de la arquitectura del procesador para la cual se generará el código.

$$\mathbf{C} \rightarrow \mathbf{IR} \rightarrow \mathbf{ARM}$$

Mejora del código

El compilador puede transformar la representación intermedia para producir un programa más eficiente sin modificar su comportamiento.

- Eliminación de operaciones innecesarias.
- Simplificación de expresiones.
- Optimización de ciclos.
- Mejor utilización de registros.

En sistemas embebidos

La optimización puede influir directamente en el tiempo de ejecución, el tamaño del firmware y el uso de memoria del microcontrolador.

Back-end del compilador

En la etapa final, el compilador transforma la representación interna del programa en instrucciones correspondientes a la arquitectura del procesador objetivo.

Para la Raspberry Pi Pico original, el código debe generarse para los núcleos ARM Cortex-M0+ del RP2040.

IR → Instrucciones ARM

Compilación cruzada

Host y Target

En sistemas embebidos utilizamos normalmente **compilación cruzada**: el compilador se ejecuta en una arquitectura diferente de aquella donde se ejecutará el programa.

PC (Host) $\longrightarrow$ **Cross-Toolchain** $\longrightarrow$ **RP2040 (Target)**

En nuestro curso

La computadora desarrolla y construye el firmware, mientras que el RP2040 constituye el sistema objetivo donde finalmente se ejecutará.

GNU Arm Embedded Toolchain

En el desarrollo para el RP2040 encontraremos herramientas identificadas mediante el prefijo:

`arm-none-eabi-`

Por ejemplo:

- `arm-none-eabi-gcc`
- `arm-none-eabi-as`
- `arm-none-eabi-ld`
- `arm-none-eabi-objcopy`

De código fuente a firmware

Cada herramienta participa en una etapa específica hasta obtener el archivo que será programado en el microcontrolador.

Toolchain

La generación de firmware es una cadena de transformaciones. El compilador constituye solo una parte de esa cadena y, a su vez, realiza varias fases internas antes de producir código para la arquitectura objetivo.

B.1.3

Compilación

¿Qué significa compilar?

Del código fuente al código objeto

La compilación es el proceso mediante el cual el código fuente escrito en un lenguaje como C se transforma en código correspondiente a la arquitectura del procesador objetivo.

En un proyecto embebido, cada archivo fuente puede compilarse de manera independiente antes de realizar el enlazado final.

`main.c` $\longrightarrow$ **Compilación** $\longrightarrow$ `main.o`

Elementos de un proyecto en C

Durante la compilación se utilizan principalmente dos tipos de archivos:

- `.c`: contienen la implementación del programa.
- `.h`: contienen declaraciones, definiciones, tipos y prototipos utilizados por otros archivos.

Ejemplo

`main.c`

`sensor.c`

`sensor.h`

Compilación de varios archivos

Compilación independiente

En un proyecto formado por varios archivos fuente, cada unidad de traducción se procesa de manera independiente y genera su propio archivo objeto.

`main.c` $\longrightarrow$ `main.o`

`gpio.c` $\longrightarrow$ `gpio.o`

`sensor.c` $\longrightarrow$ `sensor.o`

Importante

En esta etapa los archivos objeto todavía no forman un programa ejecutable completo. Deben ser combinados posteriormente por el enlazador.

Errores de compilación

Cuando el compilador no puede continuar

Si el código fuente contiene errores, el proceso de compilación se detiene y el compilador genera mensajes que permiten localizar el problema.

Ejemplo

```
main.c:15:5: error: expected ';' before 'gpio_put'
```

Errores frecuentes

- Errores de sintaxis.
- Identificadores no declarados.
- Tipos incompatibles.
- Uso incorrecto de funciones.

Warnings y Errors

No significan lo mismo

El compilador puede generar diferentes tipos de mensajes durante la construcción del programa.

- **Error**: impide generar correctamente el código objeto.
- **Warning**: señala una situación potencialmente problemática, pero puede permitir continuar la compilación.

Buena práctica

Un programa que compila no necesariamente es un programa correcto. Los *warnings* deben analizarse y no ignorarse sistemáticamente.

Arquitectura objetivo

Cuando desarrollamos para la Raspberry Pi Pico, la configuración del proyecto determina que el código generado sea compatible con la arquitectura del microcontrolador objetivo.

El sistema de construcción coordina la toolchain, las opciones de compilación y las bibliotecas necesarias del Pico SDK.

Código C $\longrightarrow$ **Código objeto** $\longrightarrow$ **RP2040**

Del código objeto al programa

¿Qué tenemos hasta ahora?

Después de la compilación disponemos de uno o varios archivos objeto que contienen código máquina, pero todavía deben integrarse entre sí y con las bibliotecas utilizadas por el proyecto.

`main.o` + `gpio.o` + **Bibliotecas** → ?

Siguiente etapa

Enlazado

B.1.4

Enlazado

¿Qué es el enlazado?

Linking

El enlazado es la etapa en la que los archivos objeto generados durante la compilación se combinan con las bibliotecas necesarias para construir un único programa ejecutable. Esta tarea es realizada por el **enlazador** (*linker*).

`main.o + gpio.o + Bibliotecas` $\longrightarrow$ **Linker** $\longrightarrow$ `programa.elf`

¿Cómo se conectan los archivos?

Los diferentes archivos objeto pueden utilizar funciones o variables definidas en otros archivos. El enlazador debe localizar estas referencias y asociarlas con su definición correspondiente.

Ejemplo

`main.o` utiliza:

`leer_sensor()`

La implementación se encuentra en:

`sensor.o`

El enlazador relaciona ambas referencias.

Código reutilizable

Durante el enlazado también se incorporan las bibliotecas utilizadas por el programa.
En nuestros proyectos, muchas de estas funciones serán proporcionadas por el **Pico SDK**.

Ejemplo

```
gpio_init()  
gpio_set_dir()  
gpio_put()  
sleep_ms()
```

¿Dónde se coloca el programa?

El enlazador también determina dónde se ubicarán las diferentes partes del programa dentro del espacio de memoria del sistema.

Entre las secciones más importantes encontramos:

- `.text`: instrucciones del programa.
- `.rodata`: datos constantes.
- `.data`: variables globales inicializadas.
- `.bss`: variables globales no inicializadas.

Mapa de memoria

El *linker script* contiene información que permite al enlazador organizar las diferentes secciones del programa dentro de las regiones de memoria disponibles en el microcontrolador.

Memoria Flash

.text

.rodata

SRAM

.data

.bss

Executable and Linkable Format

El resultado principal del proceso de enlazado es normalmente un archivo `.elf`. Este archivo contiene no solo el código máquina, sino también información sobre:

- Secciones del programa.
- Direcciones de memoria.
- Símbolos.
- Información utilizada durante la depuración.

`programa.elf`

Errores de enlazado

Compilar no significa enlazar

Un proyecto puede compilar correctamente y aun así producir errores durante el enlazado.

Ejemplo

```
undefined reference to 'leer_sensor'
```

Causas frecuentes

- Funciones declaradas pero no implementadas.
- Archivos objeto faltantes.
- Bibliotecas no incluidas.
- Símbolos definidos más de una vez.

Preparación para la programación

Después del enlazado, el archivo ELF puede transformarse en diferentes formatos según el método utilizado para programar el microcontrolador.

Para la Raspberry Pi Pico utilizaremos principalmente el formato:

.uf2

Nota

uf2 = USB Flashing Format

Enlazado

El compilador trabaja principalmente con cada unidad de traducción; el enlazador construye el programa completo.

Compilación → Archivos objeto → Enlazado → ELF

B.1.5

Programación

¿Qué significa programar un microcontrolador?

Programación del dispositivo

Programar un microcontrolador consiste en transferir el firmware generado durante el proceso de construcción hacia la memoria desde la cual será ejecutado por el dispositivo.

Firmware $\longrightarrow$ **Memoria del sistema** $\longrightarrow$ **Ejecución**

Importante

Escribir código, compilarlo y programar el dispositivo son etapas diferentes del proceso de desarrollo.

¿Dónde se almacena el firmware?

Raspberry Pi Pico

El RP2040 no incorpora memoria Flash interna para almacenar el programa. En la Raspberry Pi Pico, el firmware se almacena en una memoria Flash externa conectada al RP2040 mediante QSPI.

RP2040 $\longleftrightarrow$ **Flash QSPI**

Memoria no volátil

La memoria Flash conserva el firmware aun cuando la tarjeta deja de recibir alimentación.

USB Flashing Format

UF2 es un formato diseñado para facilitar la programación de microcontroladores mediante USB. Durante la construcción de un proyecto para Raspberry Pi Pico puede generarse un archivo `.uf2` a partir del programa enlazado.

`programa.elf` $\longrightarrow$ `programa.uf2`

Programación mediante BOOTSEL

Modo de almacenamiento masivo

Al mantener presionado el botón BOOTSEL mientras se conecta la Raspberry Pi Pico mediante USB, el RP2040 inicia su modo de arranque USB.

La computadora reconoce entonces un dispositivo de almacenamiento denominado:

RPI-RP2

Programación

El archivo .uf2 puede copiarse a esta unidad para programar la memoria Flash de la tarjeta.

¿Qué ocurre al copiar el UF2?

Proceso de programación

De forma simplificada, el proceso puede representarse como:

- 1 La Pico inicia en modo BOOTSEL.
- 2 La computadora detecta RPI-RP2.
- 3 Se transfiere el archivo .uf2.
- 4 El contenido se escribe en la memoria Flash.
- 5 El dispositivo reinicia.
- 6 Comienza la ejecución del firmware.

¿Cómo puede arrancar sin nuestro programa?

El RP2040 incorpora una memoria ROM interna que contiene código proporcionado por el fabricante para realizar funciones esenciales durante el arranque.

Entre estas funciones se encuentra el soporte necesario para iniciar el modo de programación mediante USB.

Encendido → Boot ROM → Flash → Firmware

Desarrollo y depuración

BOOTSEL no es la única forma de cargar firmware en la Raspberry Pi Pico. Durante el desarrollo también pueden utilizarse interfaces de depuración para programar y controlar la ejecución del microcontrolador.

- BOOTSEL / UF2.
- Interfaz SWD.
- Herramientas de depuración.

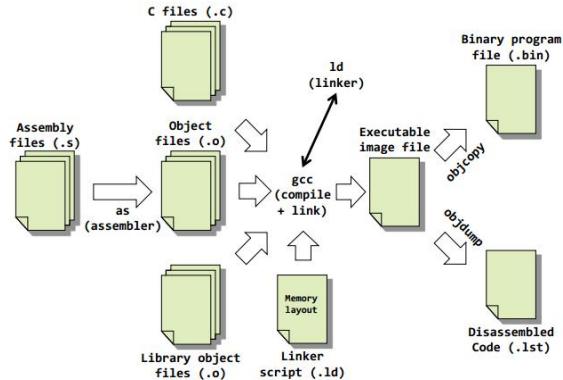
Más adelante

La interfaz SWD será retomada cuando estudiemos las herramientas de depuración del entorno de desarrollo.

Flujo completo de desarrollo

De código fuente a ejecución

Con la programación del dispositivo se completa el flujo básico de desarrollo de firmware.



Flujo de firmware

El desarrollo de firmware es un proceso compuesto por varias etapas independientes pero relacionadas:

Código fuente → **Compilación** → **Enlazado** → **Programación** → **Ejecución**

Comprender este flujo permitirá interpretar posteriormente qué realizan CMake, la toolchain y el Pico SDK durante la construcción de un proyecto.

B.2

Visual Studio Code

¿Qué es Visual Studio Code?

Entorno de desarrollo

Visual Studio Code es un editor de código extensible que permite integrar las diferentes herramientas utilizadas durante el desarrollo de firmware.

En este curso será nuestra interfaz principal de trabajo para desarrollar aplicaciones sobre la Raspberry Pi Pico.

Importante

Visual Studio Code no compila, enlaza o programa por sí mismo. Integra y permite utilizar las herramientas encargadas de realizar estas tareas.

Nuestro flujo de referencia

Desarrollo de firmware

A partir de este punto utilizaremos el siguiente flujo para identificar la función de cada herramienta del entorno de desarrollo.

Código fuente → Compilación → Enlazado → Programación → Ejecución

Pregunta fundamental

Cada vez que aparezca una nueva herramienta preguntaremos:
¿En qué etapa del flujo participa?

¿Dónde se encuentra Visual Studio Code?

Visual Studio Code

Visual Studio Code funciona como una interfaz que permite trabajar sobre prácticamente todo el flujo de desarrollo.

VS Code

Entorno integrado de trabajo

Código fuente → **Compilación** → **Enlazado** → **Programación** → **Ejecución**

Clasificación

VS Code no pertenece a una única etapa. Es la interfaz desde la cual podemos acceder y coordinar las herramientas que participan en ellas.

Elementos principales

Durante el curso utilizaremos principalmente:

- Explorador de archivos.
- Editor de código.
- Terminal integrada.
- Panel de problemas.
- Herramientas de construcción.
- Herramientas de depuración.

Ampliando las capacidades del editor

Visual Studio Code utiliza extensiones para integrar soporte para lenguajes, plataformas, sistemas de construcción y herramientas de depuración.

Durante el curso encontraremos soporte relacionado con:

- C y C++.
- CMake.
- Raspberry Pi Pico.

Importante

Una extensión puede facilitar el uso de una herramienta, pero debemos distinguir la **interfaz** de la herramienta que realmente realiza el proceso.

Estructura de un proyecto

Código fuente

Visual Studio Code permite visualizar y editar los archivos que constituyen nuestro proyecto.

```
proyecto/  
CMakeLists.txt  
main.c  
pico_sdk_import.cmake  
build/
```

Código fuente → Compilación → Enlazado → Programación → Ejecución

Etapas relacionadas

Código fuente

Construcción del proyecto

Desde VS Code

El proceso de construcción puede iniciarse desde Visual Studio Code, pero son otras herramientas las que realizan la configuración, compilación y enlazado del proyecto.

Código fuente → **Compilación** → **Enlazado** → Programación → Ejecución

Herramientas que estudiaremos

CMake + Toolchain + Pico SDK

Programación del dispositivo

Transferencia del firmware

Una vez construido el firmware, Visual Studio Code también puede integrarse con mecanismos que permiten cargar el programa en la Raspberry Pi Pico.

Código fuente → Compilación → Enlazado → **Programación** → Ejecución

Mecanismos relacionados

UF2 / BOOTSEL / SWD

Observando el programa

Después de programar el dispositivo, el firmware se ejecuta en el microcontrolador. Las herramientas de depuración permiten observar y controlar esta ejecución para analizar el comportamiento del programa.

Código fuente → Compilación → Enlazado → Programación → **Ejecución**

Herramientas relacionadas

Debugger + SWD

¿Dónde participa cada herramienta?

Herramienta	Etapas principales
Visual Studio Code	Todo el flujo como interfaz
Editor C/C++	Código fuente
Pico SDK	Código fuente / Compilación / Enlazado
CMake	Compilación / Enlazado
Compilador	Compilación
Enlazador	Enlazado
UF2 / BOOTSEL	Programación
SWD	Programación / Ejecución
Debugger	Ejecución

Visual Studio Code

Visual Studio Code es la interfaz que integra nuestro entorno de desarrollo, pero cada etapa es realizada por herramientas especializadas.

Código fuente → Compilación → Enlazado → Programación → Ejecución

A partir de ahora

Cada herramienta nueva será ubicada dentro de este flujo.

B.3

Pico SDK

¿Qué es un SDK?

Software Development Kit

Un SDK es un conjunto de herramientas, bibliotecas, archivos de cabecera y recursos que facilitan el desarrollo de software para una plataforma específica.

SDK = APIs + Bibliotecas + Headers + Build System

En nuestro curso

Utilizaremos el **Raspberry Pi Pico C/C++ SDK** para desarrollar firmware para el RP2040.

¿Dónde participa el Pico SDK?

Ubicación dentro del flujo

El Pico SDK participa principalmente durante el desarrollo del código y la construcción del firmware.

Código fuente → **Compilación** → **Enlazado** → Programación → Ejecución

Pico SDK

Código fuente + Compilación + Enlazado

¿Qué proporciona el Pico SDK?

Recursos para desarrollar firmware

El Pico SDK proporciona los elementos necesarios para desarrollar aplicaciones sin tener que construir desde cero todo el soporte del microcontrolador.

- Archivos de cabecera.
- Bibliotecas.
- APIs para acceder al hardware.
- Funciones de propósito general.
- Infraestructura de ejecución.
- Integración con el sistema de construcción.

Del registro a una API

Abstracción del hardware

El Pico SDK proporciona APIs que permiten controlar los periféricos del RP2040 mediante funciones en C, evitando que todas las operaciones tengan que realizarse directamente sobre los registros.



Importante

El SDK facilita el acceso al hardware; no reemplaza al hardware.

Bibliotecas hardware_*

El SDK proporciona APIs específicas para acceder a los diferentes periféricos disponibles en el microcontrolador.

Biblioteca	Periférico
hardware_gpio	Entradas y salidas digitales
hardware_adc	Convertidor ADC
hardware_pwm	Modulación PWM
hardware_uart	Comunicación UART
hardware_spi	Comunicación SPI
hardware_i2c	Comunicación I2C
hardware_timer	Temporización
hardware_pio	Programmable I/O

Bibliotecas pico_*

Además del acceso directo a periféricos, el SDK incluye bibliotecas que proporcionan funcionalidades de mayor nivel.

Biblioteca	Función
pico_stdlib	Funcionalidades comunes
pico_time	Temporización y alarmas
pico_sync	Sincronización
pico_multicore	Uso de múltiples núcleos
pico_unique_id	Identificación del dispositivo

Ejemplo: GPIO

Utilizando el SDK

Para controlar un GPIO podemos utilizar funciones proporcionadas por las APIs del Pico SDK.

Código

```
#include "pico/stdlib.h"

gpio_init(15);
gpio_set_dir(15, GPIO_OUT);
gpio_put(15, 1);
```

Código fuente → Compilación → Enlazado → Programación → Ejecución

¿Qué es pico/stdlib.h?

Punto de entrada común

El archivo de cabecera `pico/stdlib.h` proporciona acceso a funcionalidades utilizadas frecuentemente en aplicaciones desarrolladas con el Pico SDK.

Ejemplos

```
gpio_init()  
gpio_put()  
sleep_ms()  
stdio_init_all()
```

Más adelante

Conforme estudiemos cada periférico utilizaremos también sus APIs específicas, como `hardware/adc.h`, `hardware/pwm.h` o `hardware/i2c.h`.

El SDK y la compilación

Construcción del firmware

Los archivos de cabecera y el código proporcionado por el SDK deben estar disponibles durante la construcción del proyecto.

El sistema de construcción configura qué componentes del SDK serán utilizados para generar nuestra aplicación.

Código fuente → **Compilación** → Enlazado → Programación → Ejecución

Herramientas relacionadas

Pico SDK + CMake + Toolchain

Incorporando bibliotecas

Las bibliotecas utilizadas por nuestra aplicación deben incorporarse al programa durante el proceso de enlazado.

Por ejemplo, un proyecto puede solicitar la biblioteca:

`pico_stdlib`

Compilación → **Enlazado** → Programación → Ejecución

¿Cómo indicamos qué necesita nuestro proyecto?

El Pico SDK utiliza **CMake** para gestionar el proceso de construcción.

Por ejemplo, podemos indicar que nuestro programa debe enlazarse con `pico_stdlib`:

CMakeLists.txt

```
add_executable(mi_programa
    main.c
)

target_link_libraries(mi_programa
    pico_stdlib
)
```

Siguiente sección

B.4 CMake

Relación entre herramientas

Elemento	Responsabilidad
Visual Studio Code	Interfaz de desarrollo
Pico SDK	APIs y bibliotecas
CMake	Configuración de la construcción
Toolchain	Compilación y enlazado
UF2 / SWD	Programación
Debugger	Análisis de la ejecución

Pico SDK

El Pico SDK constituye la interfaz de software que utilizaremos para desarrollar aplicaciones y acceder a los recursos del RP2040.

Código fuente → **Compilación** → **Enlazado** → Programación → Ejecución

Pico SDK

B.4

CMake

¿Qué es CMake?

Sistema de construcción

CMake es una herramienta que permite describir cómo debe construirse un proyecto de software. A partir de archivos de configuración, CMake organiza los archivos fuente, dependencias, bibliotecas y opciones necesarias para construir el firmware.

Importante

CMake no es un compilador.

¿Dónde participa CMake?

Ubicación dentro del flujo

CMake interviene principalmente en la preparación y coordinación del proceso de construcción que conduce a la compilación y al enlazado.

Código fuente → **Compilación** → **Enlazado** → Programación → Ejecución

CMake

Configura y coordina Compilación + Enlazado

¿Por qué necesitamos CMake?

Un proyecto puede crecer rápidamente

Un programa sencillo puede contener únicamente un archivo `main.c`, pero un proyecto real puede incorporar:

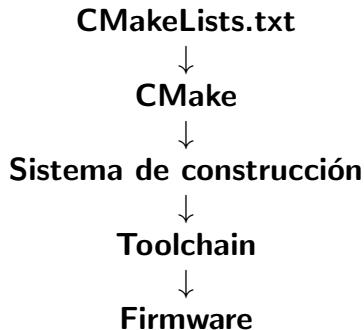
- Múltiples archivos `.c` y `.h`.
- Bibliotecas del Pico SDK.
- Bibliotecas desarrolladas por el usuario.
- Diferentes opciones de compilación.
- Dependencias entre componentes.

Problema

Compilar y enlazar manualmente todos estos elementos se vuelve cada vez más complejo conforme crece el proyecto.

Separación de responsabilidades

CMake determina qué debe construirse y genera la configuración necesaria para que las herramientas correspondientes realicen el trabajo.



Archivo de configuración

El archivo `CMakeLists.txt` describe cómo debe construirse nuestro proyecto.
En él podemos indicar:

- Nombre y características del proyecto.
- Archivos fuente.
- Bibliotecas necesarias.
- Dependencias.
- Opciones de construcción.

`CMakeLists.txt`

Proyecto para Raspberry Pi Pico

Un archivo CMakeLists.txt sencillo puede contener los siguientes elementos:

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)
include(pico_sdk_import.cmake)
project(mi_proyecto C CXX ASM)
pico_sdk_init()
add_executable(mi_programa
    main.c
)
target_link_libraries(mi_programa
    pico_stdlib
)
pico_add_extra_outputs(mi_programa)
```

Definición del proyecto

Configuración inicial

```
cmake_minimum_required(VERSION 3.13)
```

```
include(pico_sdk_import.cmake)
```

```
project(mi_proyecto C CXX ASM)
```

```
pico_sdk_init()
```

¿Qué estamos indicando?

Estas instrucciones establecen la configuración inicial del proyecto e incorporan el soporte necesario del Pico SDK al sistema de construcción.

Definiendo el ejecutable

```
add_executable()
```

```
add_executable(mi_programa  
    main.c  
)
```

Función

Esta instrucción crea un objetivo de construcción denominado `mi_programa` e indica los archivos fuente que forman parte de él.

`main.c` → `mi_programa`

Proyecto con varios archivos

Escalando el proyecto

Cuando nuestra aplicación crezca, podemos incorporar varios archivos fuente al mismo objetivo.

Ejemplo

```
add_executable(mi_programa
    main.c
    sensores.c
    control.c
)
```

`main.c + sensores.c + control.c` → **Aplicación**

```
target_link_libraries()
```

```
target_link_libraries(mi_programa  
    pico_stdlib  
)
```

Función

Esta instrucción especifica las bibliotecas que deben estar disponibles para construir el objetivo. En este ejemplo utilizamos `pico_stdlib`, una biblioteca fundamental del Pico SDK.

Compilación → **Enlazado** → Firmware

Agregando periféricos

Bibliotecas específicas

Cuando utilicemos determinados periféricos del RP2040 podremos incorporar las bibliotecas correspondientes.

Ejemplo

```
target_link_libraries(mi_programa
    pico_stdlib
    hardware_adc
    hardware_pwm
)
```

pico_stdlib
hardware_adc
hardware_pwm

Funciones comunes
Convertidor ADC
Generador PWM

```
pico_add_extra_outputs()
```

```
pico_add_extra_outputs(mi_programa)
```

Función

Esta función del Pico SDK permite generar formatos adicionales del programa construido, entre ellos los archivos utilizados para programar la Raspberry Pi Pico.

ELF $\longrightarrow$ BIN HEX UF2

¿Dónde aparecen los archivos generados?

Es una práctica común realizar la construcción en un directorio separado denominado build.

```
mi_proyecto/  
    CMakeLists.txt  
    main.c  
    pico_sdk_import.cmake  
    build/  
        mi_programa.elf  
        mi_programa.uf2
```

Configurar y construir

Dos acciones diferentes

El proceso puede dividirse conceptualmente en dos pasos:

1. Configurar

```
cmake -S . -B build
```

2. Construir

```
cmake --build build
```

Importante

Durante la construcción se ejecutan las herramientas necesarias para compilar y enlazar el proyecto.

¿Quién hace realmente el trabajo?

No confundir las herramientas

Herramienta	Responsabilidad
VS Code	Interfaz de desarrollo
Pico SDK	APIs, bibliotecas y soporte
CMake	Configura la construcción
Compilador	Genera código objeto
Enlazador	Construye el ejecutable
UF2 / SWD	Permiten la programación
RP2040	Ejecuta el firmware

CMake dentro del flujo

Código fuente → **Compilación** → **Enlazado** → Programación → Ejecución

CMake



Configura y coordina la construcción

Idea clave

CMake describe **qué debe construirse**; la toolchain realiza las operaciones necesarias para transformar el código fuente en firmware.

B.5

Organización de proyectos

¿Por qué organizar un proyecto?

De un ejemplo a una aplicación

Los primeros programas pueden desarrollarse utilizando únicamente un archivo `main.c`. Sin embargo, conforme aumenta la complejidad del firmware, mantener todo el código en un solo archivo dificulta su comprensión, mantenimiento y reutilización.

`main.c` $\longrightarrow$ **Proyecto modular**

Objetivo

Separar el programa de acuerdo con las responsabilidades de cada componente.

¿Dónde participa la organización?

Ubicación dentro del flujo

La organización del proyecto comienza en el código fuente, pero también determina qué archivos serán procesados durante la compilación y posteriormente integrados durante el enlazado.

Código fuente → **Compilación** → **Enlazado** → Programación → Ejecución

Organización del proyecto

Código fuente + Compilación + Enlazado

Programa en un solo archivo

Estructura inicial

En programas pequeños es común concentrar toda la aplicación en un único archivo.

```
mi_proyecto/  
  CMakeLists.txt  
  main.c
```

¿Qué ocurre cuando crece?

El archivo comienza a mezclar inicialización, lectura de sensores, control de actuadores, comunicaciones y lógica de la aplicación.

Programación modular

Separación de responsabilidades

La programación modular consiste en dividir el firmware en componentes más pequeños, donde cada módulo realiza una tarea específica.

`main.c`



`sensor control comunicacion`

Ventaja

Cada módulo tiene una responsabilidad definida.

Implementación e interfaz

Una forma común de construir un módulo en C consiste en separar su interfaz de su implementación.

Archivo	Contenido principal
sensor.h	Declaraciones y prototipos
sensor.c	Implementación de las funciones

Interfaz $\longleftrightarrow$ **Implementación**

Ejemplo de un módulo

sensor.h

```
#ifndef SENSOR_H
#define SENSOR_H
    void sensor_init(void);
    float sensor_read(void);
#endif
```

sensor.c

```
#include "sensor.h"
void sensor_init(void) {
    // Inicializacion
}
float sensor_read(void) {
    // Lectura
    return 0.0f;
}
```

Utilizando un módulo

Desde la aplicación

El programa principal puede utilizar las funciones públicas del módulo incluyendo su archivo de cabecera.

main.c

```
#include "pico/stdlib.h"
#include "sensor.h"

int main() {

    sensor_init();

    while (true) {
        float value = sensor_read();
    }
}
```

Estructura del proyecto

Separación por directorios

Cuando el proyecto crece, podemos organizar los archivos en directorios de acuerdo con su función.

```
mi_proyecto/  
  CMakeLists.txt  
  src/  
    main.c  
    sensor.c  
    control.c  
  include/  
    sensor.h  
    control.h  
  build/
```

¿Qué colocamos en cada directorio?

Elemento	Responsabilidad
src/	Implementaciones .c
include/	Interfaces y cabeceras .h
build/	Archivos generados
CMakeLists.txt	Configuración de construcción

Importante

El directorio build/ contiene archivos generados durante la construcción y debe mantenerse separado del código fuente.

Organización y CMake

CMake debe conocer los archivos

Después de organizar el código, debemos indicar al sistema de construcción qué archivos forman parte de nuestra aplicación.

CMakeLists.txt

```
add_executable(mi_programa
    src/main.c
    src/sensor.c
    src/control.c
)

target_include_directories(mi_programa PRIVATE
    ${CMAKE_CURRENT_LIST_DIR}/include
)
```

Separación por niveles

En un sistema embebido no todo el código tiene la misma responsabilidad. Podemos separar la lógica de la aplicación del código utilizado para interactuar con el hardware.

Aplicación



API



Driver



Hardware

¿Qué es una API?

Application Programming Interface

Una **API** es una interfaz de programación que define las funciones y operaciones disponibles para que un componente de software pueda utilizar los servicios proporcionados por otro.

Ejemplo

```
sensor_init()  
  
sensor_read()  
  
sensor_set_mode()
```

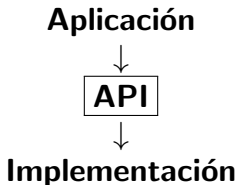
Idea clave

La API establece **cómo utilizamos** un componente, sin necesidad de conocer todos los detalles de su implementación.

API: interfaz e implementación

Separación

La API especifica las operaciones disponibles, mientras que la implementación contiene el código encargado de realizar esas operaciones.



En C

<code>sensor.h</code>	→	Interfaz pública
<code>sensor.c</code>	→	Implementación

¿Qué es un Driver?

Controlador de dispositivo

Un **driver** es un componente de software encargado de controlar o proporcionar acceso a un dispositivo o periférico de hardware.

El driver traduce las operaciones solicitadas por la aplicación en operaciones específicas sobre el hardware.

Aplicación → Driver → Hardware

Ejemplo de un Driver

Pantalla OLED

Supongamos que conectamos una pantalla OLED mediante el protocolo I2C. Podemos desarrollar un driver que encapsule los detalles necesarios para controlar el dispositivo.

Driver

`ssd1306.h`

`ssd1306.c`

Aplicación → Driver SSD1306 → Pico SDK → I2C → OLED

API y Driver no son lo mismo

	API	Driver
¿Qué es?	Interfaz de software	Componente de software
¿Define?	Cómo utilizar un servicio	Cómo controlar un dispositivo
¿Hardware?	No necesariamente	Generalmente relacionado con hardware
Ejemplo	<code>sensor_read()</code>	<code>ssd1306.c</code>

Relación

Un driver puede proporcionar una **API** para que la aplicación pueda utilizar el dispositivo sin conocer los detalles de bajo nivel.

Encendiendo un LED

Cuando utilizamos una función del Pico SDK podemos observar los diferentes niveles involucrados.

Aplicación → `gpio_put(15,1)` → **API del Pico SDK**

→ **Acceso al periférico GPIO** → **Registros / Hardware RP2040**

→ **Pin GPIO**

```
mi_proyecto/  
  CMakeLists.txt  
  src/  
    main.c  
    control.c  
  include/  
    control.h  
  drivers/  
    ssd1306.c  
    ssd1306.h  
  build/
```

Organización

Los drivers pueden mantenerse separados de la lógica principal de la aplicación, facilitando su reutilización y mantenimiento.



Importante

Estas capas representan diferentes niveles de abstracción. Dependiendo del problema, el programador puede trabajar en niveles más altos o acceder directamente a niveles más cercanos al hardware.

Organización dentro del flujo

Código fuente → **Compilación** → **Enlazado** → Programación → Ejecución

Elementos organizados

Aplicación → API → Driver → Pico SDK → Hardware

Idea clave

Organizar un proyecto no consiste únicamente en crear carpetas. Implica separar responsabilidades y definir interfaces claras entre los diferentes componentes del firmware.

B.6

Depuración

¿Qué es la depuración?

Debugging

La depuración es el proceso de identificar, analizar y corregir errores en el comportamiento de un programa.

En sistemas embebidos, algunos errores solamente pueden observarse cuando el firmware se encuentra ejecutándose sobre el hardware real.

Importante

Compile correctamente no garantiza que el programa funcione correctamente.

¿Dónde ocurre la depuración?

Ubicación dentro del flujo

La depuración se relaciona principalmente con la ejecución del firmware. Algunas herramientas también permiten programar directamente el microcontrolador.

Código fuente → Compilación → Enlazado → **Programación** → **Ejecución**

Depuración

Principalmente durante la **ejecución**.

No todos los errores son iguales

Durante el desarrollo podemos encontrar errores en diferentes etapas del flujo.

Tipo	Ejemplo
Sintaxis	Falta de un punto y coma
Compilación	Tipo de dato incompatible
Enlazado	Función no encontrada
Ejecución	Acceso incorrecto a memoria
Lógico	Resultado diferente al esperado
Hardware	Configuración incorrecta de un periférico

Código fuente → Errores de sintaxis → **Compilación** → Errores de compilación

→ **Enlazado** → Errores de enlazado → **Ejecución** → Errores lógicos y de ejecución

¿Qué es un Debugger?

Herramienta de depuración

Un *debugger* es una herramienta que permite controlar y observar la ejecución de un programa. Puede detener temporalmente el procesador y permitir al programador analizar el estado interno del sistema.

Programa en ejecución



Debugger

¿Qué podemos observar?

Estado interno

Durante una sesión de depuración podemos observar información sobre el estado del programa y del procesador.

- Variables.
- Punto actual de ejecución.
- Registros del procesador.
- Memoria.
- Pila de llamadas.
- Estado de algunos periféricos.

Objetivo

Determinar qué está haciendo realmente el programa.

Breakpoints

Puntos de interrupción

Un *breakpoint* permite detener la ejecución cuando el procesador alcanza un punto determinado del programa.

Ejemplo

```
while (true) {  
  
    valor = sensor_read();    // Breakpoint  
  
    control_update(valor);  
}
```

Al detenerse

Podemos inspeccionar el estado del programa en ese instante.

Control de la ejecución

Después de detener el programa podemos avanzar controladamente a través del código.

Operación	Función
Continue	Continuar ejecución
Pause	Detener temporalmente
Step Over	Ejecutar la siguiente línea
Step Into	Entrar en una función
Step Out	Salir de la función actual
Restart	Reiniciar el programa

Estado del programa

Mientras el procesador está detenido podemos observar los valores de las variables y analizar cómo cambian durante la ejecución.

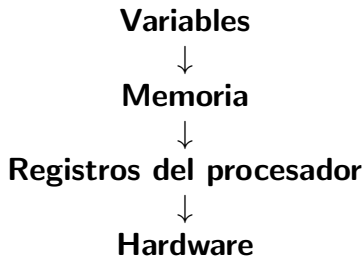
Ejemplo

referencia	=	25.0
temperatura	=	28.4
error	=	-3.4
salida	=	0.62

Inspección de memoria y registros

Más cerca del hardware

Las herramientas de depuración también permiten observar información asociada directamente con el procesador y la memoria.



Importante

Esto permite relacionar el código escrito en C con el estado real del microcontrolador.

¿Cómo depuramos un microcontrolador?

Existen diferentes estrategias

Dependiendo del hardware disponible y del tipo de problema, podemos utilizar diferentes métodos para analizar la ejecución del firmware.

1. Instrumentación

2. Debug Probe

3. Otra Pico como Debug Probe

Método 1: Instrumentación

Observar sin detener

Una estrategia sencilla consiste en incorporar instrucciones de diagnóstico dentro del propio programa.

El firmware envía información sobre variables, estados o eventos mientras continúa ejecutándose.



Instrumentación mediante printf()

Observando variables

Podemos utilizar la función `printf()` para enviar información desde el microcontrolador.

Ejemplo

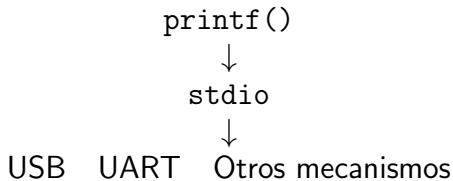
```
printf("ADC = %d\n", valor_adc);  
printf("Temperatura = %.2f\n", temperatura);  
printf("Error = %.2f\n", error);
```

Programa → `printf()` → Computadora

Salida estándar

stdio

El Pico SDK proporciona mecanismos de entrada y salida estándar que permiten dirigir información hacia diferentes interfaces.



Durante el curso

Utilizaremos principalmente la salida mediante **USB**.

Ejemplo con Pico SDK

Salida de diagnóstico

```
#include <stdio.h>
#include "pico/stdlib.h"

int main() {

    stdio_init_all();

    while (true) {
        printf("Sistema ejecutandose\n");
        sleep_ms(1000);
    }
}
```

Resultado

Los mensajes pueden visualizarse desde una terminal.

`printf()` no es un debugger completo

La instrumentación es muy útil para observar información, pero normalmente no proporciona las capacidades de una depuración hardware.

- No detiene automáticamente el procesador.
- No permite ejecución paso a paso.
- Debemos decidir previamente qué información mostrar.
- Los mensajes pueden modificar los tiempos de ejecución.

Método 2: Debug Probe

Depuración mediante hardware dedicado

Una *Debug Probe* es un dispositivo que funciona como interfaz entre la computadora y el puerto de depuración del microcontrolador.

En la Raspberry Pi Pico podemos utilizar la interfaz **SWD**.

Computadora



Debug Probe



Pico

¿Qué es SWD?

Serial Wire Debug

SWD es una interfaz de depuración utilizada por procesadores ARM que permite establecer comunicación con el sistema objetivo.

Puede utilizarse para programar el microcontrolador y controlar su ejecución durante una sesión de depuración.

Debug Probe



RP2040

Señales principales

SWDIO

SWCLK

GND

Raspberry Pi Debug Probe

Sonda dedicada

Raspberry Pi dispone de una Debug Probe diseñada para proporcionar una interfaz entre USB y SWD. También puede funcionar como puente entre USB y UART.

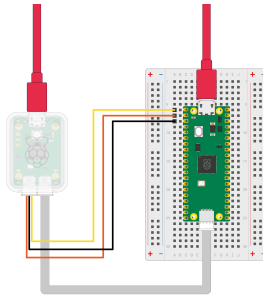
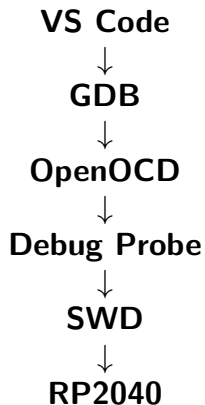


Figura: Raspberry Pi Debug Probe conectada por SWD a una Raspberry Pi Pico

Software y hardware trabajando juntos

La depuración mediante SWD requiere diferentes componentes que trabajan conjuntamente.



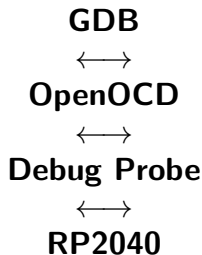
GNU Debugger

GDB es el debugger que permite controlar la ejecución del programa y consultar su estado. Entre otras operaciones permite:

- Establecer breakpoints.
- Continuar o detener la ejecución.
- Ejecutar paso a paso.
- Inspeccionar variables.
- Consultar memoria.

Open On-Chip Debugger

OpenOCD proporciona la comunicación entre las herramientas de depuración de la computadora y la interfaz de depuración del microcontrolador.



Método 3: Una Pico como Debug Probe

Otra posibilidad

No es obligatorio utilizar una Debug Probe dedicada.

Una Raspberry Pi Pico puede ejecutar un firmware especial que permite utilizarla como interfaz de depuración para otra Raspberry Pi Pico.

Pico Debugger



Pico Objetivo

Transformando una Pico

Raspberry Pi proporciona el firmware debugprobe, que permite convertir una Pico en una interfaz:

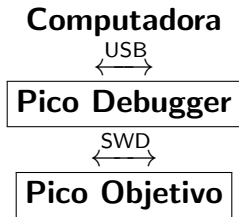
USB ↔ SWD

y

USB ↔ UART

Configuración

Una tarjeta ejecuta el firmware debugprobe y funciona como herramienta de depuración. La segunda tarjeta ejecuta la aplicación que deseamos analizar.



Conexión entre las tarjetas

Interfaz SWD

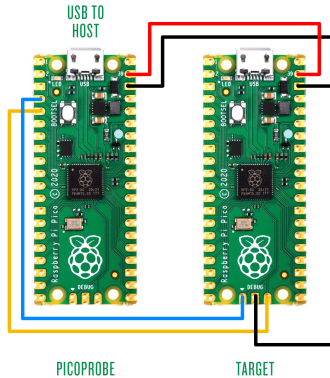
La Pico utilizada como debugger se comunica con la Pico objetivo mediante las señales de la interfaz SWD.

Debugger		Objetivo
SWDIO	↔	SWDIO
SWCLK	→	SWCLK
GND	↔	GND

Importante

Ambos sistemas deben compartir una referencia de tierra.

Depuración con dos Pico



PC $\xleftrightarrow{\text{USB}}$ Pico Debugger $\xleftrightarrow{\text{SWD}}$ Pico Objetivo

¿Qué podemos hacer con este método?

Depuración hardware

Al utilizar una segunda Pico como Debug Probe podemos disponer de las herramientas propias de una sesión de depuración.

- Breakpoints.
- Pause y Continue.
- Step Into.
- Step Over.
- Step Out.
- Inspección de variables.
- Inspección de memoria.
- Inspección de registros.

Comparación de los tres métodos

	Instrumentación	Debug Probe	Segunda Pico
Hardware adicional	No	Sí	Otra Pico
printf()	Sí	Sí	Sí
Breakpoints	No	Sí	Sí
Paso a paso	No	Sí	Sí
Variables	Mediante mensajes	Inspección directa	Inspección directa
SWD	No	Sí	Sí

¿Cuál utilizaremos?

Durante el curso

Utilizaremos los métodos de acuerdo con las necesidades de cada práctica.

Diagnóstico cotidiano

`printf()` → USB → Terminal

Depuración hardware

PC → Pico Debugger → SWD → Pico Objetivo

Debug Probe dedicada

Conoceremos su funcionamiento como herramienta profesional, aunque podemos reproducir gran parte del flujo utilizando una segunda Raspberry Pi Pico.

Método 1

Firmware → `printf()` → USB/UART → Terminal

Método 2

VS Code → GDB/OpenOCD → Debug Probe → SWD → Pico

Método 3

VS Code → GDB/OpenOCD → Pico Debugger → SWD → Pico Objetivo

Código fuente → Compilación → Enlazado → **Programación** → **Ejecución**

Idea clave

Depurar significa observar y analizar el comportamiento real del firmware. Podemos hacerlo mediante instrumentación o mediante un debugger capaz de controlar directamente la ejecución del microcontrolador.

Observar → Identificar → Corregir